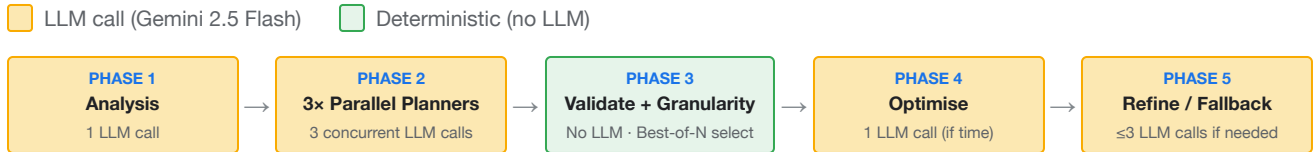


Gemini-Powered Multi-Agent DAG Scheduler

MLSys 2026 Competition · Track B: Agent Reasoning

OVERVIEW

We present a five-phase multi-agent pipeline that schedules memory-constrained DAG workloads using **Gemini 2.5 Flash**. The central design principle separates concerns cleanly: the LLM is responsible solely for *which* operations to group into subgraphs, while all constraint-sensitive decisions—granularity selection, memory validation, and latency calculation—are handled by deterministic Python. This prevents the model from generating memory-invalid schedules and eliminates the need for it to guess numeric thresholds.



AGENT ARCHITECTURE

Phase 1 — Problem Analysis. A dedicated `AnalyzerAgent` receives the raw problem JSON enriched with pre-computed graph metrics from NetworkX: critical-path length, per-node fan-in/fan-out, MatMul chain identification, and topological levels. The free-text analysis is forwarded verbatim to all downstream planners, ensuring they share a common structural understanding before proposing schedules.

Phase 2 — Parallel Strategy Planning. Three instances of `PlannerAgent` run concurrently via `asyncio.gather`, each receiving an identical analysis context but a different *strategy persona*:

- **Aggressive:** maximise operation fusion; accept sub-native granularity and recomputation to minimise subgraph count.
- **Balanced:** follow the critical path at native granularity; apply Split-K for MatMul chains.
- **Conservative:** prefer native granularity and spilling; group only when operations trivially co-fit.

Output token budgets scale with problem size as `min(65536, max(8192, num_ops × 1000))`, and Gemini's extended thinking is disabled (`thinking_budget=0`) for all JSON calls. On large problems (e.g., 32 ops), internal reasoning would otherwise consume most of the token budget, leaving insufficient capacity for the full JSON output and causing truncation.

Phase 3 — Deterministic Validation and Granularity Search. Every candidate partition passes through `optimize_granularities`, which finds the *largest* valid `[w, h, k]` triplet for each subgraph via a structured search over power-of-two candidates—removing any burden from the LLM for this numerically sensitive decision. A pruning step (`_feasible_retentions`) discards any `tensors_to_retain` entry whose full tensor size exceeds scratchpad capacity, preventing working-set overflows on large-tensor problems. The valid candidate with the lowest total latency is selected as the best solution.

Phase 4 — Optimisation. `OptimizerAgent` proposes improved traversal orders (e.g., snake patterns for MatMul to exploit data reuse) and refined retention decisions. The result passes through deterministic validation before replacing the current best only if it achieves lower latency.

Phase 5 — Iterative Refinement with Guaranteed Fallback. If no valid candidate was produced, `RefinerAgent` receives the invalid schedule alongside specific validation errors from our deterministic validator (e.g., *"Step 2: working set 1,065,088 bytes exceeds capacity 250,000 bytes"*). Up to three correction iterations are attempted. If all fail, a deterministic fallback—one operation per subgraph with the algorithmically optimal granularity—guarantees a valid submission within the 10-minute time limit.

Module	Responsibility	LLM?
<code>core/dag_utils.py</code>	NetworkX graph metrics (critical path, fan-out/in, MatMul chains)	No
<code>core/validator.py</code>	Full constraint checking (dependency order, memory, coverage)	No
<code>core/granularity.py</code>	Algorithmic search for the largest valid <code>[w, h, k]</code>	No
<code>core/latency.py</code>	Accurate latency model incl. split-K asymmetric passes	No
<code>agents/analyzer.py</code>	Deep structural analysis enriched with pre-computed metrics	Yes ×1
<code>agents/planner.py</code>	Three parallel planners with distinct strategy personas	Yes ×3
<code>agents/optimizer.py</code>	Traversal order and retention fine-tuning	Yes ×1
<code>agents/refiner.py</code>	Targeted error correction from validator diagnostics	Yes ≤3

PROMPTING STRATEGY

Constraint-first framing. All system prompts state memory limits numerically upfront and embed a step-by-step reasoning framework: (1) analyse dependencies, (2) compute the working set for each proposed grouping, (3) select granularity, (4) decide on retention. This chain-of-thought structure reduces the rate at which the model produces constraint-violating schedules.

Strategy personas. Instead of prompting for a single "best" schedule, we issue three concurrent prompts with explicit strategy labels and trade-off priorities. This increases the probability that at least one candidate is valid and near-optimal across diverse graph topologies, with the best-of-N selection acting as a lightweight ensemble.

Error-as-context refinement. During Phase 5, the refiner receives the prior invalid schedule alongside structured diagnostics from our deterministic validator. Providing specific, machine-generated error messages (exact step index, actual vs. allowed working set, violated dependency) produces more targeted corrections than generic retry instructions.

JSON-only output discipline. All planning and optimisation calls use `response_mime_type: "application/json"` with `thinking_budget=0`. This ensures the full output token budget is available for JSON generation and prevents response truncation on large problems.

RESULTS ON RELEASED BENCHMARKS

The baseline is a deterministic one-op-per-subgraph schedule with the algorithmically optimal granularity per step—equivalent to the conservative lower bound for each problem.

Problem	Ops	Fast Memory	Baseline Latency	Agent Latency	Agent Subgraphs	Speedup
example_problem	2	20,000	6,554	3,277	1	2.00×
mlsys-2026-1	5	60,000	891,290	642,253	3	1.39×
mlsys-2026-5	19	30,000	4,971,998	4,971,998	19	1.00×
mlsys-2026-9	32	250,000	198,642,237	27,261,358	8	7.29×
mlsys-2026-13	63	600,000	22,226,862	22,226,862	63	1.00×
mlsys-2026-17	103	500,000	5,081,851	5,081,851	103	1.00×

The agent's largest gains arise from operation fusion: on **mlsys-2026-9** (32 MatMul-heavy ops on 1024×1024 tensors), the aggressive planner collapsed 32 individual subgraphs into 8, reducing redundant slow-memory round trips and achieving a **7.3× latency reduction**. On memory-tight problems (mlsys-2026-5, fast memory = 30 KB), tensor sizes preclude grouping at any granularity above the native floor, so the agent matches the baseline exactly.

SUBMISSION DETAILS

Entry point (`agent.py`): accepts two positional arguments — `python3 agent.py <input.json> <output.json>` — and reads `GOOGLE_API_KEY` from the injected environment variable. All network traffic goes exclusively to `generativelanguage.googleapis.com` via the `google-genai` SDK; no other external endpoints are contacted.

Agent modules (`agents/`): `base.py` — shared async Gemini client with retry logic and JSON extraction; `analyzer.py`, `planner.py`, `optimizer.py`, `refiner.py` — one file per pipeline phase. `orchestrator.py` coordinates all phases, enforces the 9.5-minute internal deadline via `asyncio.wait_for`, and guarantees a valid output by falling back to the deterministic scheduler if all LLM phases fail or time out.

Core library (`core/`): pure-Python, no LLM. `dag_utils.py` — NetworkX graph analysis; `validator.py` — full constraint checking; `granularity.py` — optimal `[w, h, k]` search and retention pruning; `latency.py` — split-K aware latency model.

Prompts (`prompts/`): `system_analysis.txt`, `system_planning.txt`, `system_optimization.txt`, `system_refinement.txt`, and `few_shot_examples.json` (annotated scheduling examples for in-context learning). All files are loaded at runtime and injected verbatim into each LLM call.

Dependencies (`requirements.txt`): `google-genai`, `networkx`, `python-dotenv`, `rich`. Runtime: Python 3.12.